Software Stack Supplement

The Open Software Stack Across All Repositories of Naziru Halilu

Languages and libraries and techniques and tools found by reading the code of every repository

Naziru Halilu halilunaziru73@gmail.com github.com/halilunaziru73-creator

Contents

1 Purpose and how this was made 2

2 The repositories at a glance 2

3 Languages and formats 4

4 Python packages used in all repositories 5

5 Techniques 6

6 Tools 9

7 Installing the Python packages of the account 9

8 What is not claimed 10

S1 Purpose and how this was made

This document is the companion of Supplementary Material 5 of the SWAFL series, widened from one repository to the whole account. Every public repository of the account (17 in all: SWAFL and 14 other research repositories, the personal website, and the profile repository) was copied, and its files, its dependency lists (`requirements.txt`, `pyproject.toml`, `environment` files, PyInstaller `.spec` files, GitHub workflow files) and its code were read. The Python packages come from the import lines of the code (scripts and notebooks), not from memory; the techniques come from the methods stated in each README and from the names of the functions and classes in the code. The software of the SWAFL repository itself is described in full in Supplementary Material 5 of that repository and is only summarised here.

Words used. A *language* is the way instructions are written. A *library* or *package* is code written by others that the code calls for one job; a package is a library wrapped so that one command installs it (`pip install`). A *program* or *tool* is a finished application that you start. A *technique* is a method of analysis. Counts are of files and lines on the day of writing; a package is listed as used only when the code imports it.

S2 The repositories at a glance

Table 1: The 17 repositories, what each is, and its main languages and libraries.

Repository	What it is	Languages (lines)	Main libraries and tools
Coupled Water Fluxes (SWAFL)	SWAFL Desktop 1.1.0 and the four articles	JavaScript (14 100) and Python (about 10 000 over the folders) and LaTeX (103 files) and Markdown	Electron and Leaflet and GeoTIFF.js and SheetJS and numpy and matplotlib and pandas and scipy and scikit-learn and statsmodels and Plotly; Daisy 7.1.14 (C++)
Personal website	the personal website and the CVs	HTML and CSS (3 800)	Google Fonts; PDF files of the CVs; GitHub Pages
Profile page	the profile page	Markdown and HTML	GitHub profile README
Operationalizing GIS and Machine Learning	three field campaigns in one validation framework	Python (920)	numpy and pandas and scipy and scikit-learn and matplotlib and Pillow
Zaria Crop ET and Irrigation DSS	crop evapotranspiration and irrigation decision support	Python (7 200) and JSON	Tkinter and numpy and pandas and matplotlib and python-docx and Pillow and PyInstaller
NaCROP	the Nigeria version of the same crop and irrigation program	Python (9 600) and JSON	Tkinter and numpy and pandas and matplotlib and python-docx and Pillow and PyInstaller
Gully Biocontrol Digital Twin	a four-layer digital twin of a gully	Python (3 300)	numpy and pandas and scipy and scikit-learn and matplotlib and joblib and Pillow; setuptools; GitHub Actions

Repository	What it is	Languages (lines)	Main libraries and tools
GACL	a contrastive learning framework for agronomic images	Python (4 600) and a notebook	PyTorch and torchvision and scikit-image and scikit-learn and pandas and openpyxl and PyQt6 and Pillow and matplotlib and scipy
Hybrid CNN BiLSTM Attention	a hybrid deep network for sediment transport	Python (1 560)	numpy and pandas and scikit-learn and matplotlib (the network itself is written with numpy)
RGB Proxy Vegetation Indexing	a desktop program for crop image analysis	Python (6 800)	PyQt6 and PyTorch and torchvision and scikit-image and scikit-learn and numpy and pandas and openpyxl and Pillow and matplotlib and rasterio; PyInstaller
Livestock Slurry Suitability	suitability mapping for slurry application	Python (1 130) and QGIS project files	rasterio and geopandas and shapely and pyproj and fiona and PyKrige and scikit-learn and TensorFlow with Keras and pandas and numpy and scipy and matplotlib; QGIS
Dairy Processing DSS	a decision-support model for a dairy plant	Python (1 640)	pandas and numpy and scikit-learn and scipy and matplotlib and pytest and flake8; GitHub Actions
MSRK	a GeoAI framework for soil and yield maps	Python (1 640)	scikit-learn and PyKrige and scipy and numpy and pandas and matplotlib
Geospatial Data Analysis	a six-module course in GIS and remote sensing	Jupyter notebooks (2 600 lines of code) and some JavaScript	rasterio and geopandas and shapely and rasterstats and folium and OWSLib and GDAL (`osgeo`) and scikit-learn and matplotlib and pandas and numpy; Google Earth Engine
Vineyard Ripening Brix	NDVI and sugar monitoring of a vineyard	QGIS project and style files	QGIS 4 (project and style and metadata files); Sentinel-2 NDVI; spreadsheets
Grape Moth Pest	trap monitoring of a vine pest	QGIS project and style files	QGIS 4; digital elevation model; shapefiles and GeoPackage
STADOS	atmospheric correction of Sentinel-2 NDVI	Python (410) and a notebook	numpy and pandas and scipy and matplotlib; Jupyter

S3 Languages and formats

Table 2: All languages and formats found in the 17 repositories, with their role.

Language or format	Where	Role and why it is used
Python	11 repositories besides SWAFL (38 800 lines of Python and 2 960 lines of notebook code in them)	the language of the analysis and the models and the desktop programs. Why: the libraries for data and maps and learning are mature, and the scripts run unchanged on any computer.
Jupyter notebooks (`.ipynb`)	the course repository and STADOS and GACL	step by step analysis that shows code and results together. Why: a reader can run and follow each step.
JavaScript	SWAFL and the course repository	the program SWAFL and the interactive pages. Why: it runs in a browser and in Electron without a compiler.
HTML and CSS	the website and the graphical abstracts and the SWAFL window	the pages that people see. Why: opens in any browser with no installation.
LaTeX and BibTeX	the SWAFL repository	articles and supplements with exact equations.
Markdown	every repository (`README.md`)	the explanation of each repository. Why: plain text that GitHub shows as a page.
JSON	most repositories	configuration and data and results that both people and programs read.
YAML (`.yml`)	workflow files	the instructions of GitHub Actions.
TOML (`pyproject.toml`)	the digital twin and the dairy model	how a Python package is described and built. Why: a package can then be installed by name.
CSV and XLSX	most research repositories	tables of data and results. Why: any spreadsheet program opens them.
GIS formats: shapefile (`.shp` with `.shx .dbf .prj .cpg`) and GeoPackage and GeoTIFF (`.tif`) and KML and QGIS project (`.qgz`) and QGIS style (`.qml`) and QGIS metadata (`.qmd`)	the GIS repositories	maps and rasters and styles. Why: open standards that QGIS and Python read.
Word documents (`.docx`)	manuscripts and reports	the text of manuscripts for co-authors.
Archives (`.7z`) and model files (`.joblib`)	the course and the digital twin	large data and trained models.
PyInstaller `.spec`	NaCROP and the crop DSS and the image program	the recipe that turns a Python program into one Windows file.
Batch files (`.bat`)	NaCROP and the crop DSS	start files for Windows.

S4 Python packages used in all repositories

Table 3: Every third-party Python package imported by the code, with its role.

Package (import name)	Repositories	Role and why it is used
numpy	12 repositories	arrays and the numerical core. Why: every other package builds on it.
pandas	12	tables of observations and results. Why: grouping and joining by name avoids index errors.
matplotlib	12	every figure and map layout. Why: one library gives print quality figures.
scipy	8	statistics and curve fitting and image shifting and interpolation.
scikit-learn (`sklearn`)	9	random forest and gradient boosting and ridge and elastic net and PCA and k-nearest neighbours and k-means and cross validation. Why: standard and repeatable learners.
PyTorch (`torch`) and torchvision	GACL and the image program	deep networks (hypergraph transformer and ResNet101 and Faster R-CNN). Why: flexible training of new architectures.
TensorFlow and Keras	slurry suitability	a multilayer perceptron and a patch-based convolutional network. Why: fast prototyping of small networks.
scikit-image (`skimage`)	GACL and the image program	texture features (grey-level co-occurrence matrix) and image filters.
PyKrige (`pykrige`)	slurry suitability and MSRK	kriging and regression kriging. Why: geostatistical interpolation without writing the variogram code.
rasterio	6 (in some it is optional)	reading and writing rasters (GeoTIFF). Why: the standard Python door to GDAL.
GeoPandas and Shapely and pyproj and Fiona	slurry suitability and the course	vector data (points and lines and polygons) and map projections.
rasterstats	the course	zonal statistics of rasters inside polygons.
folium and OWSLib and animatplot	the course	web maps and web map services and animated plots.
GDAL (`osgeo`)	the course	raster and vector conversion.
Pillow (`PIL`)	6	reading and saving images and the icons of the programs.
openpyxl	GACL and the image program (listed in the requirements and used by pandas)	lets pandas read Excel files.
joblib	the digital twin and GACL	saving and loading trained models.
python-docx (`docx`)	NaCROP and the crop DSS	writing the Word farm reports.
xlrd	NaCROP and the crop DSS	reading old Excel files (optional).

Package (import name)	Repositories	Role and why it is used
PyQt6	GACL and the image program	the GIS-style window of the desktop program.
Tkinter (standard library)	NaCROP and the crop DSS and the image program	the simple desktop window. Why: it ships with Python.
pytest and flake8	the dairy model	unit tests and style checks.
PyInstaller	NaCROP and the crop DSS and the image program	builds the standalone Windows program. Why: the user then needs no Python.
setuptools	the digital twin and the dairy model	packages the code so it installs by name.
IPython and Jupyter	the course and STADOS	running the notebooks.

The standard library is also used (for example `dataclasses` in 23 files, `argparse` in 12, and `threading`). Modules written by the author inside a repository (for example `bgdt`, `gacl`, `nacrop`, `zaria_maize`, `dairy_dss`, `msrk_engine`, `qgis_ui`, `core`, `cartohelpers`, `hybrid_model`) are not third-party packages and are not listed.

S5 Techniques

Table 4: The techniques used, where, and why.

Technique	Where it is used	Role and why
Process simulation with Daisy	SWAFL (10 190 runs)	the soil and plant and atmosphere model whose runs the suite reads.
Evapotranspiration equations (FAO-56 Penman-Monteith and ASCE and Priestley-Taylor and Makkink and Turc and Hargreaves-Samani and Thornthwaite and Blaney-Criddle and Dalton)	the crop DSS and NaCROP	ten ways to estimate crop water use, compared on the same weather.
Soil water balance and irrigation scheduling (TAW and RAW depletion)	the crop DSS and NaCROP and SWAFL	when and how much to irrigate.
Crop growth in four steps (canopy cover and transpiration and biomass and yield)	NaCROP	yield from water use.

Technique	Where it is used	Role and why
Pedotransfer and hydraulic functions (Saxton and Rawls and van Genuchten and Mualem)	SWAFL	soil hydraulics from texture.
Hydraulic design (Hazen and Williams and Manning and volume balance)	SWAFL	pipe and surface irrigation design.
Bayesian data assimilation and hydrological and erosion modelling (HEC-HMS and HEC-RAS 2D and RUSLE-LISEM named in the README)	the gully digital twin	combines sensors and models into a twin that can be asked what-if questions.
Digital twin architecture (physical and digital and brain and service layers)	the gully digital twin	a structure that joins measurement and prediction.
Neural network written from scratch (Conv1D and bidirectional LSTM and attention pooling and dense layers with a gradient check)	the sediment transport repository	the layers are coded with numpy so that every step can be inspected.
Model explanation (permutation importance and kernel SHAP and Monte Carlo dropout and bootstrap intervals)	the sediment transport and the digital twin and the dairy model	shows which inputs matter and how sure the result is.
Contrastive and hypergraph transformer learning	GACL	learns image features that do not depend on camera geometry.
Classical image analysis (RGB proxy vegetation indices and NDVI and NDRE and grey-level co-occurrence texture and PCA and k-nearest neighbours and k-means)	the image program and GACL	plant condition from ordinary photographs.
Pretrained deep detectors (ResNet101 and Faster R-CNN)	the image program	optional detection in images.

Technique	Where it is used	Role and why
Regression kriging with random forest trend	slurry suitability and MSRK and the validation framework	a data-driven trend plus geostatistical interpolation of the residuals.
Local singularity and multifractal indices	MSRK	a multiscale feature for each location.
Multicriteria overlay (GIS-MCDA)	slurry suitability	the conventional suitability map that the new methods are compared with.
Deep surrogates (multilayer perceptron and patch-based CNN)	slurry suitability	continuous suitability maps.
Dark-object subtraction and temporal harmonisation	STADOS	corrects the atmosphere in multi-temporal Sentinel-2 NDVI.
Optical and radar remote sensing (Sentinel-1 and Sentinel-2) and supervised and unsupervised classification	the course	land cover maps.
Terrain analysis (altitude and slope and aspect from a digital elevation model)	grape moth and slurry	shows where conditions differ.
Trap monitoring and mating-disruption costing	grape moth	pest pressure by place and the cost of control.
Logarithmic regression forecast of sugar (Brix) against day of year	vineyard ripening	predicts the harvest date.
Machine learning model comparison (ridge and elastic net and random forest and gradient boosting) with leave-one-out validation	the dairy model	picks a profit model that suits few data.
Linear programming and Monte Carlo simulation	the dairy model	allocates capacity among products and shows risk.
Tiered validation (reproducibility and mapping and association and prediction and decision and transfer)	the validation framework	states how far each result can be trusted.

S6 Tools

Table 5: Tools and services used to build and publish the repositories.

Tool	Where	Role and why
QGIS 4 (the 4.0 release named Norrköping in the project files)	the vineyard and slurry repositories	the desktop GIS in which the maps were built; the project and style and metadata files are in the repository.
Google Earth Engine	the course	cloud processing of satellite archives.
Sentinel-1 and Sentinel-2 imagery	the course and STADOS and the vineyard repositories	the satellite data analysed.
Jupyter and nbconvert	the course and STADOS	notebooks and their HTML export.
PyInstaller	three desktop programs	builds one Windows file with no Python needed.
GitHub Actions	8 repositories	a syntax check of all Python files on every push (Python 3.11) in seven, and a test and style and coverage run on Python 3.10 and 3.11 and 3.12 in the dairy model.
GitHub Pages and `.nojekyll`	the website and the graphical abstracts	publishes the pages from the repository. Why: free hosting that needs no server.
Zenodo and DOI	the articles	permanent archives of data and code.
Citation files (`CITATION.cff`)	14 repositories	tells readers how to cite the work.
Google Fonts	the website and graphical abstracts	the typefaces (IBM Plex and Space Grotesk and Playfair Display and others).
Electron and electron-builder and Node.js	SWAFL	the desktop shell and the portable Windows file (see Supplementary Material 5).
pandoc and XeLaTeX and BibTeX	SWAFL	the manual and the articles.
Playwright	SWAFL	tests of the real program.
Word and Excel formats	manuscripts and reports	files for co-authors and reviewers.

S7 Installing the Python packages of the account

The package names that `pip` uses differ from the import names for four of them: scikit-learn (imported as `sklearn`), Pillow (`PIL`), python-docx (`docx`) and PyTorch (`torch`, installed as `torch`).

```
python3 -m pip install numpy pandas matplotlib scipy scikit-learn joblib pillow openpyxl
python3 -m pip install scikit-image torch torchvision PyQt6 tensorflow
python3 -m pip install pykrige python-docx xlrd pytest flake8 pyinstaller jupyter
```

The geospatial packages depend on GDAL and install most reliably from conda-forge, as the requirement files of the repositories advise.

```
conda install -c conda-forge geopandas rasterio fiona shapely pyproj rasterstats \
    folium owslib gdal
```

QGIS is a separate program and is installed from https://qgis.org. Each repository also has its own `requirements.txt`, which lists only what that repository needs.

S8 What is not claimed

Versions are not stated for packages whose requirement files give only a minimum or no version, because no exact version is recorded. Two repositories carry the map files of one QGIS project each and no code. The GIS repositories were made in QGIS, so their method is recorded in the project and style files and the README, not in scripts. The names of the physical models in the gully twin (HEC-HMS and HEC-RAS 2D and RUSLE-LISEM) are those named in its README; their programs are not part of the repository.